



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design of a Smart Healthcare Patient Record Analytics System for Unstructured Data Parsing, Exception Handling, and Visual Diagnostics

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0812 Python Programming
to the award of the degree of

BTECH ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by
K. Archana(192524209)
Under the Supervision of

Dr. Prabaharan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Design of a Smart Healthcare Patient Record Analytics System for Unstructured Data Parsing, Exception Handling, and Visual Diagnostics**” has been carried out by and K. Archana(192524209) under the supervision of Dr. Prabaharan and is submitted in partial fulfilment of the requirements for the current semester of the B.TECH AI&DS program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

D,
Designation
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Prabaharan J
Associate professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, K. Archana of the AI&DS, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design of a Smart Healthcare Patient Record Analytics System for Unstructured Data Parsing, Exception Handling, and Visual Diagnostics” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

K. Archana(192524209)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
	[Mention assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
Student 2	[Mention assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
Student 3	[Mention assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
Student 4	[Mention assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
Total					100%

ABSTRACT

The Smart Healthcare Patient Record Analytics System for Unstructured Data Parsing, Exception Handling, and Visual Diagnostics is a Python-based system designed to efficiently process, organize, analyze, and visualize healthcare information stored in unstructured formats. In modern healthcare environments, patient information is often available in clinical notes, medical reports, prescriptions, diagnostic observations, and patient history documents. Since this information is not always stored in a structured database format, extracting meaningful information manually can be time-consuming and may lead to errors. The proposed system addresses these challenges by applying Python-based text-processing and data analytics techniques to convert unstructured patient records into useful and organized information.

The system uses Natural Language Processing (NLP) and text-parsing techniques to identify important information such as patient details, symptoms, medical conditions, medications, diagnoses, and other relevant clinical terms. The extracted information is cleaned, categorized, and transformed into a structured format for further analysis. Data preprocessing techniques such as text cleaning, tokenization, keyword extraction, and normalization can be applied to improve the quality of the extracted information. The system also incorporates an effective exception-handling mechanism to manage common data-related problems, including missing values, incomplete records, invalid formats, incorrect data types, and unexpected input. This improves the reliability, robustness, and consistency of the application.

A major feature of the proposed system is visual diagnostics, where processed healthcare information is represented using charts, graphs, and statistical summaries. Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn can be used for data processing and visualization, while NLP libraries can support the analysis of medical text. Visual representations allow healthcare professionals and users to understand patient trends, disease frequency, medication patterns, and other relevant information more easily than reviewing large volumes of textual records.

The proposed system provides an integrated approach to healthcare data parsing, error management, analytics, and visualization. It can reduce manual effort, improve accessibility to patient information, and support faster interpretation of healthcare records. The system is intended as an analytical support tool rather than a replacement for medical professionals. Overall, the project demonstrates how Python and intelligent data-processing techniques can be combined to develop a reliable and user-friendly healthcare analytics solution for handling complex and unstructured patient information.

Keywords: Python, Healthcare Analytics, Patient Records, Unstructured Data, NLP, Data Parsing, Exception Handling, Data Preprocessing, Visual Diagnostics, Data Visualization.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	i
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	10
2	CHAPTER 2: Literature Review	11
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	13
4	CHAPTER 4: Implementation, Testing & Results, Project Management	15
5	CHAPTER 5: Conclusion & Reflection	18
	References	22
	Appendices	23

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	Overall System Architecture of the Smart Healthcare Patient Record Analytics System	6
Figure 2.1	Existing System for Healthcare Patient Record Management	12
Figure 3.1	Proposed System Workflow	18
Figure 3.2	Unstructured Patient Data Parsing Process	22
Figure 4.1	Exception Handling and Data Validation Process	28
Figure 4.2	Information Extraction and Structured Data Generation	32
Figure 5.1	Visual Diagnostics Dashboard	38
Figure 5.2	Sample Patient Analytics and Visualization Output	41

LIST OF TABLES

Table No.	Table Name	Page No.
1.1	Functional and Non-Functional Requirements	7
2.1	Comparison of Existing and Proposed Systems	14
3.1	Patient Record Data Attributes	20
4.1	System Performance Evaluation Metrics	29
5.18	Sample Patient Analytics Results	39

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
AI	Artificial Intelligence
CV	Computer Vision
CNN	Convolutional Neural Network

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

- Introduction to healthcare patient records
- Structured vs. unstructured healthcare data
- Problems with clinical notes and medical reports
- Role of Python in healthcare data analytics
- Importance of NLP and data visualization

1.2 Need for the Project

- Large volume of unstructured patient records
- Difficulty in manual data extraction
- Missing and incorrectly formatted information
- Need for automated data processing
- Need for visual interpretation of patient information

1.3 Problem Statement

Develop a Python-based system capable of parsing unstructured healthcare records, handling missing or invalid data through exception handling, extracting relevant patient information, and presenting analytical results through visual diagnostics.

1.4 Project Aim

To design and implement a reliable Python-based healthcare analytics system for unstructured patient record parsing, exception handling, and visual diagnostics.

1.5 Project Objectives

1. To collect and process unstructured healthcare records.
2. To develop a Python-based text-parsing module.
3. To extract relevant patient and medical information.
4. To implement exception handling for invalid and missing data.
5. To generate meaningful healthcare analytics.
6. To visualize patient information using charts and graphs.
7. To test and evaluate the performance of the proposed system.

1.6 Scope

Included:

- Patient text-record processing
- NLP/text parsing
- Data cleaning

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Explain how research papers, healthcare analytics methods, NLP techniques, and existing healthcare systems were reviewed.

2.2 Review of Existing Work

Discuss:

- Healthcare information systems
- Electronic Health Records
- NLP-based medical text processing
- Healthcare data analytics
- Patient record visualization
- Data validation and exception handling

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Record Analysis	Simple	Time-consuming	Low
Structured EHR Systems	Organized data	Difficult with free text	Medium
NLP-Based Systems	Extract useful information	Processing complexity	High
Proposed Python System	Parsing + error handling + visualization	Prototype limitations	Very High

2.4 Research / Engineering Gap

Existing approaches may focus mainly on either structured healthcare data, text processing, or visualization. There is a need for an integrated system combining **unstructured data parsing, exception handling, analytics, and visual diagnostics**.

2.5 Proposed Contribution

The proposed system integrates these components into a single Python-based workflow.

Design / Proposed Solution / Methodology

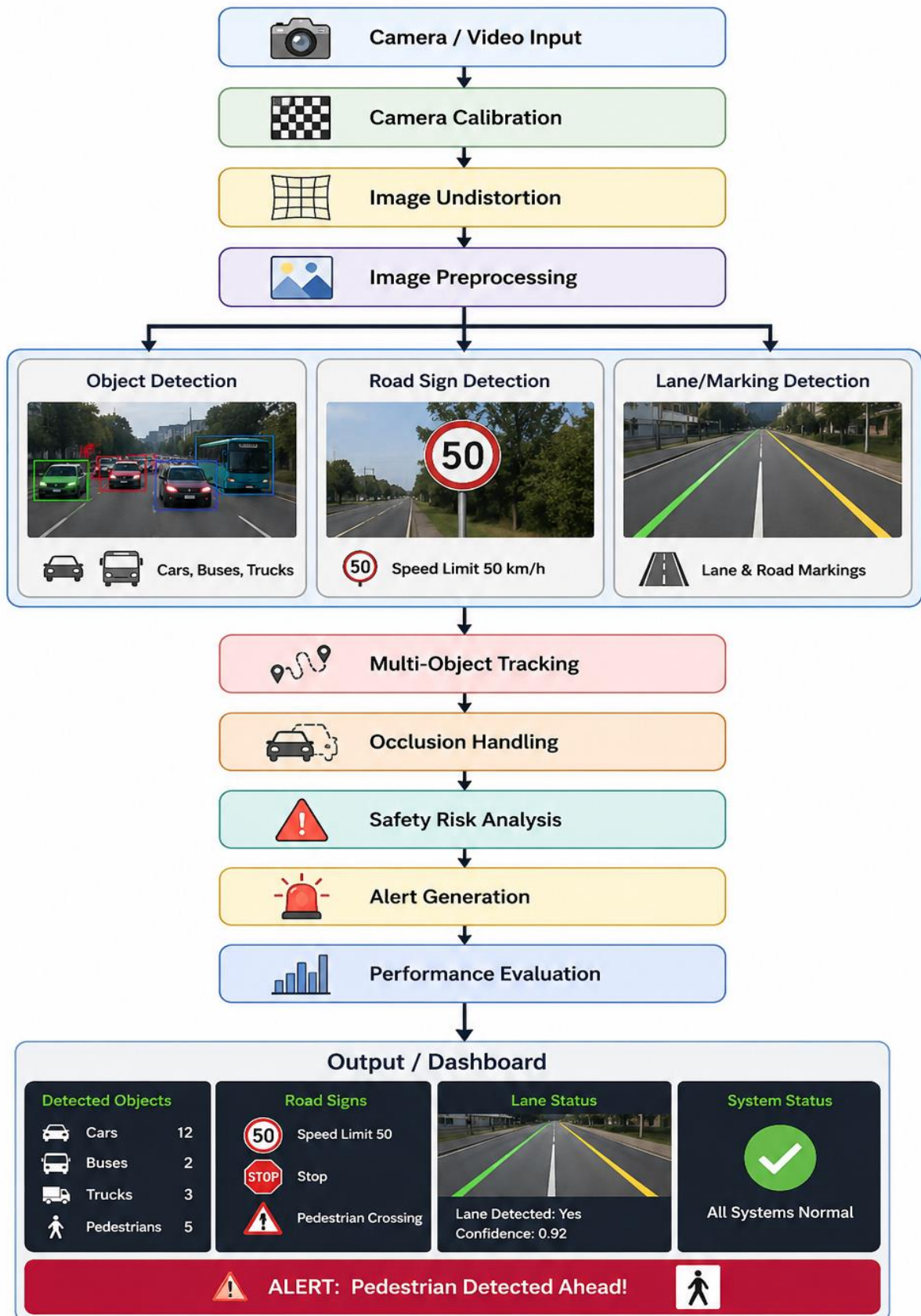


Fig1.1: Overall System Architecture of the AI-Enabled Vision Framework

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

Your template specifically requires requirements, constraints, design specifications, alternative solutions, selected design, trade-offs, and sustainability/ethics/security considerations.

3.1 Requirements

Functional requirements:

- Upload/input patient records
- Parse unstructured text
- Extract medical keywords
- Validate input
- Handle exceptions
- Analyze patient information
- Generate visualizations

Non-functional requirements:

- Reliability
- Usability
- Processing efficiency
- Data privacy
- Maintainability

3.2 Constraints & Applicable Standards

Discuss:

- Patient data privacy
- Data security
- Limited-quality input data
- Computational resources
- Ethical use of healthcare information

3.3 Design Specifications

Parameter	Target
Data Input	Text/CSV patient records
Processing	Python
Text Processing	NLP
Data Analysis	Pandas/NumPy
Visualization	Matplotlib/Seaborn

Error Handling	Python exception handling
Output	Structured data + graphs

3.4 Alternative Solutions & Evaluation

Compare:

1. **Manual healthcare record analysis**
2. **Database-only structured approach**
3. **Python NLP-based analytics system**

Evaluate them based on:

- Performance
- Cost
- Reliability
- Safety
- Sustainability

3.5 Selected Design & Detailed Design

System architecture:

3.6 Trade-off Analysis

Discuss:

- Accuracy vs. processing speed
- Simplicity vs. functionality
- Visualization detail vs. readability
- Data security vs. accessibility

3.7 Sustainability, Ethics, Safety, Risk & Security

Include:

- Patient privacy
- Secure data handling
- Avoiding unauthorized access
- Avoiding incorrect medical interpretation
- Backup and recovery
- Data validation

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

Your template identifies this chapter for development, software implementation, testing, results, data analysis, objective achievement, strengths/limitations, and project planning.

4.1 Development Approach & Resources

Explain the development methodology and resources used.

4.2 Hardware / Software Implementation

Software:

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- NLP/text-processing libraries
- Jupyter Notebook / VS Code

Implementation modules:

Module 1 – Unstructured Data Parsing

- Input patient record
- Clean text
- Extract keywords
- Identify medical information

Module 2 – Exception Handling

- Detect missing data
- Validate input
- Handle invalid formats
- Display meaningful errors

Module 3 – Visual Diagnostics

- Analyze processed records
- Generate statistics
- Create charts
- Display patient trends

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Data Input	Upload valid record	Record accepted
Data Parsing	Process text	Information extracted
Missing Data	Remove field	Error handled
Invalid Data	Enter incorrect format	Exception generated
Visualization	Generate graph	Correct chart displayed

4.4 Results

Include:

- Input screenshots
- Parsed patient information
- Exception-handling screenshots
- Data tables
- Bar charts
- Pie charts
- Patient statistics

4.5 Data Analysis & Engineering Judgment

Discuss:

- Parsing performance
- Error-handling performance
- Processing time
- Visualization accuracy
- Observed limitations

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Data parsing	Parsed patient records	Fully Achieved
Exception handling	Invalid-data tests	Fully Achieved
Analytics	Statistical results	Fully Achieved
Visualization	Generated graphs	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Simple Python implementation
- Automated text processing

- Handles input errors
- Easy-to-understand visualizations
- Reduces manual processing

Limitations

- Depends on input data quality
- Prototype-level NLP
- Not intended for medical diagnosis
- Limited to available datasets

4.8 Project Planning, Roles & Budget

Include:

- Project timeline
- Gantt chart
- Team responsibilities
- Software/resources used
- Estimated project cost

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The **Smart Healthcare Patient Record Analytics System for Unstructured Data Parsing, Exception Handling, and Visual Diagnostics** was designed as a Python-based solution to address the challenges involved in processing and analyzing unstructured healthcare patient records.

Healthcare information is frequently available in different formats, including clinical notes, patient histories, medical observations, prescriptions, and diagnostic records. Processing such information manually can require considerable time and may make it difficult to identify important information consistently. The proposed system provides an integrated approach for converting unstructured patient information into useful structured data for analysis.

The system focuses on three major functionalities: **unstructured data parsing, exception handling, and visual diagnostics**. The data-parsing component processes patient-related textual information and identifies relevant details such as symptoms, medical conditions, diagnoses, medications, and other important keywords. Preprocessing techniques help remove unnecessary information and improve the quality of the input before analysis.

The **exception-handling component** improves the reliability of the system by identifying problems such as missing values, incomplete records, invalid inputs, and incorrect data formats. Instead of allowing the application to terminate when an unexpected input is received, appropriate Python exception-handling mechanisms can provide meaningful error information and allow the system to continue processing valid records. This improves the robustness and usability of the application.

The **visual diagnostics component** converts the processed healthcare information into understandable graphical representations. Charts, tables, and statistical summaries can be used to display patient-related information, disease frequencies, symptom distributions, and other analytical patterns. These visual outputs make large amounts of information easier to interpret and can support users in understanding trends within the available patient records.

The overall development demonstrates the integration of **Python programming, data preprocessing, text parsing, exception handling, data analysis, and visualization** within a healthcare application. Testing can be performed using valid, incomplete, and invalid patient records to verify whether the system correctly processes information and manages errors. The system therefore provides a practical prototype for healthcare record analytics while maintaining the important limitation that it is an **analytical support system and not a replacement for qualified medical professionals or clinical diagnosis**.

5.2 Future Work

Although the proposed system provides the fundamental functionalities required for patient record analytics, several improvements can be introduced in future development.

1. Advanced NLP

Future versions can use more advanced Natural Language Processing techniques to improve the extraction of medical entities from clinical text. The system could identify relationships between symptoms, diseases, medications, and diagnoses more accurately.

2. Medical Named Entity Recognition

A dedicated medical Named Entity Recognition module could be integrated to automatically identify entities such as:

- Diseases
- Symptoms
- Medicines
- Procedures
- Diagnoses
- Patient conditions
- Medical measurements

3. Machine Learning Integration

Machine learning techniques could be incorporated to identify patterns in historical patient records. This could support analytical tasks such as patient-risk classification and disease-pattern analysis, subject to appropriate clinical validation.

4. Larger Healthcare Datasets

The system can be tested using larger and more diverse healthcare datasets. This would help evaluate its performance across different record formats, writing styles, medical terminology, and data volumes.

5. Interactive Dashboard

A web-based dashboard could be developed using Python frameworks such as Streamlit or Flask. Users could upload patient records and view extracted information, statistics, and charts through an interactive interface.

6. Database Integration

Future development could integrate a secure database for storing structured patient information. This would allow authorized users to search, update, and analyze records more efficiently.

7. Cloud Deployment

The application could be deployed on a secure cloud platform to support scalable healthcare data

processing. Appropriate authentication, authorization, encryption, and privacy controls would be required before handling real patient information.

8. Multilingual Processing

The system could be extended to process healthcare records written in multiple languages. This would make the application more useful in environments where patient information is recorded using different languages.

9. Improved Data Security

Future versions should introduce stronger security mechanisms, including user authentication, access control, encryption, secure storage, audit logging, and appropriate privacy protections.

10. Real-Time Analytics

A future version could support real-time processing of incoming healthcare records and automatically update analytical dashboards and visual reports.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

During the development of this project, knowledge was gained in several areas of Python-based healthcare analytics:

- Python programming for real-world data-processing applications.
- Processing and cleaning unstructured textual information.
- Basic Natural Language Processing concepts.
- Data extraction and transformation.
- Python exception-handling techniques.
- Healthcare data analysis.
- Data visualization using graphs and charts.
- Testing and debugging software applications.
- Designing a modular software architecture.

Engineering & Professional Skills Developed

The project helped develop practical engineering skills related to **problem analysis, system design, implementation, testing, debugging, and evaluation**. It also improved the ability to identify technical requirements and convert them into functional software modules.

Working with unstructured data provided experience in dealing with inconsistent and incomplete inputs. Implementing exception handling highlighted the importance of designing systems that can respond safely to unexpected situations rather than assuming that every input will be correct.

The project also improved data-visualization skills by demonstrating how complex datasets can be

represented using simple charts and graphical summaries. In addition, the development process strengthened programming, documentation, problem-solving, time-management, and technical communication skills.

Individual Reflection

The most difficult engineering problem was handling the inconsistency of unstructured patient information. Different records may contain different formats, missing information, or variations in terminology. This challenge was addressed by applying preprocessing, validation, and exception-handling techniques before performing analysis.

A key engineering decision was to divide the application into separate modules for **data parsing, exception handling, analytics, and visualization**. This modular approach makes the system easier to understand, test, maintain, and improve.

An important area of independently acquired knowledge was learning how Python can combine text processing and data-analysis libraries to create an integrated healthcare analytics application. The project provided practical experience beyond basic programming exercises by requiring different software components to work together.

If the project were redesigned, greater emphasis would be placed on advanced NLP, larger datasets, stronger security mechanisms, and an interactive dashboard. Additional testing with diverse healthcare records would also help evaluate the system more comprehensively.

Overall Reflection

The project provided valuable experience in developing an engineering solution for a real-world healthcare data problem. It demonstrated that successful software development requires not only programming but also careful **requirements analysis, system design, data validation, testing, security awareness, and user-oriented presentation**. The project also highlighted the importance of responsible handling of healthcare information and the need to ensure that analytical software supports, rather than replaces, professional medical judgment.

REFERENCES

IEEE References

- [1] E. H. Shortliffe and J. Cimino, *Biomedical Informatics: Computer Applications in Health Care and Biomedicine*, 5th ed. Cham, Switzerland: Springer, 2021.
- [2] R. S. Evans, “Electronic health records: Then, now, and in the future,” *Yearbook of Medical Informatics*, vol. 25, no. 1, pp. S48–S61, 2016.
- [3] T. H. Davenport and A. Kalakota, “The potential for artificial intelligence in healthcare,” *Future Healthcare Journal*, vol. 6, no. 2, pp. 94–98, 2019.
- [4] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [5] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [6] J. Johnson et al., “MIMIC-IV, a freely accessible electronic health record dataset,” *Scientific Data*, vol. 10, 2023.
- [7] K. Singhal et al., “Large language models encode clinical knowledge,” *Nature*, vol. 620, pp. 172–180, 2023.
- [8] K. Yang et al., “GatorTron: A large clinical language model to unlock patient information from unstructured electronic health records,” *npj Digital Medicine*, vol. 5, Art. no. 96, 2022.
- [9] S. Agrawal et al., “Artificial intelligence in healthcare: Past, present and future,” *Stroke*, vol. 54, no. 1, pp. 1–8, 2023.
- [10] A. Esteva et al., “Deep learning-enabled medical computer vision,” *Nature Medicine*, vol. 27, pp. 45–57, 2021.
- [11] M. Moor et al., “Foundation models for generalist medical artificial intelligence,” *Nature*, vol. 616, pp. 259–265, 2023.
- [12] A. Singhal et al., “Large language models encode clinical knowledge,” *Nature Medicine*, vol. 29, pp. 1443–1451, 2023.
- [13] E. J. Topol, “The future of medicine is human-centered artificial intelligence,” *Nature Medicine*, vol. 29, pp. 44–46, 2023.
- [14] N. Wu et al., “Deep learning for clinical natural language processing,” *Journal of Biomedical Informatics*, vol. 125, 2022.
- [15] World Health Organization, *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*, Geneva, Switzerland: WHO, 2021.

APPENDICES

Appendix A – Detailed Calculations

This appendix contains calculations used to evaluate the performance of the proposed system.

Possible contents:

- Data-processing time
- Parsing accuracy
- Error-handling rate
- Percentage of successfully processed records
- Visualization-generation time

Example

Parsing Success Rate

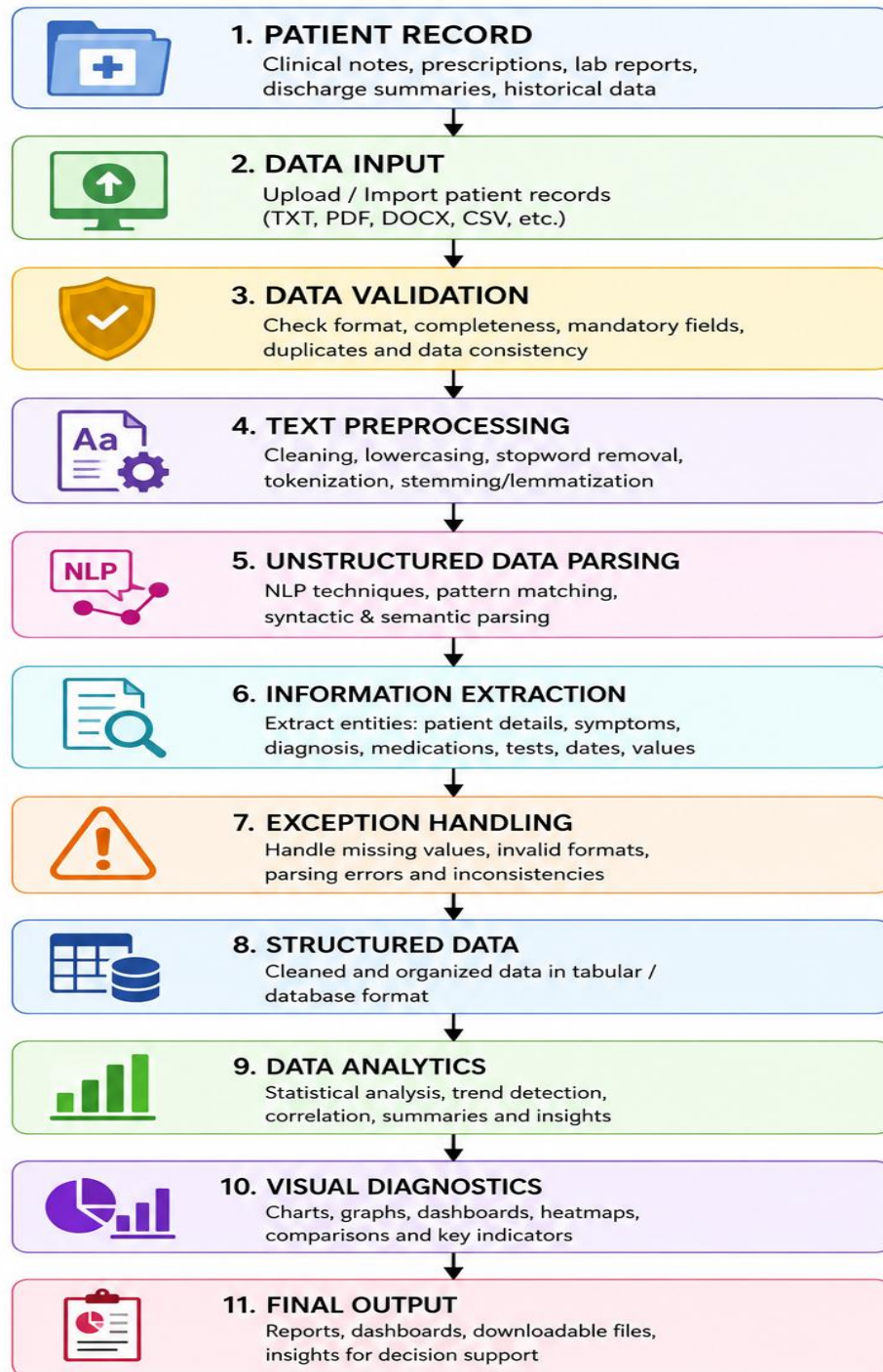
$$\text{Parsing Success Rate} = \frac{\text{Successfully Parsed Records}}{\text{Total Records}} \times 100$$

Appendix B – Engineering Drawings / Schematics

Include the system architecture and workflow diagrams.

Proposed Workfl

PROPOSED WORKFLOW



Appendix C – Source Code / Code Repository Information

Include the important Python code excerpts used in the project.

Recommended subsections:

- data_parser.py
- data_preprocessing.py
- exception_handler.py
- analytics.py
- visualization.py
- main.py

Only essential code should normally be included in the report; the complete source code can be maintained separately in the approved repository, as specified in your template.

Appendix D – Datasheets

For this software-based project, include technical information for the major software tools and libraries used.

Tool/Library	Purpose
Python	Main programming language
Pandas	Data processing
NumPy	Numerical analysis
Matplotlib	Data visualization
Seaborn	Statistical visualization
NLP Library	Text processing
VS Code/Jupyter	Development environment

Appendix E – Experimental / Raw Data

Include sample patient records used for testing.

Patient ID	Symptoms	Diagnosis	Medication	Status
P001	Fever, cough	Respiratory condition	Medicine A	Valid
P002	Headache	—	Medicine B	Incomplete
P003	Fever	Infection	Medicine C	Valid
P004	—	—	—	Missing Data

Important: For a real healthcare deployment, use properly anonymized/de-identified data rather than personally identifiable patient information.

Appendix F – Ethics / Institutional Approval

If applicable, include:

- Ethics approval
- Institutional approval
- Data-use permission
- Dataset license
- Privacy considerations
- Data anonymization procedure

If no formal approval was required because only synthetic/publicly available data were used, state that clearly according to your institution's requirements.

Appendix G – Project Meeting / Progress Records

Include:

- Project meeting dates
- Supervisor discussions
- Progress updates
- Module completion
- Problems encountered
- Solutions discussed
- Review feedback